DATA SCIENCE / GUIA TÉCNICO EP

NumPy

Arrays, eixos, broadcasting e cálculo reproduzível

Objetivo. Estudo aprofundado para apoiar os notebooks de Programação Python, com explicações, exemplos executáveis, exercícios e cuidados para ambientes reais.

Mapa de estudo. 1. Arrays, shape, dtype e eixos • 2. Views, copies e indexação • 3. Broadcasting e máscaras • 4. Aleatoriedade, precisão e qualidade • 5. Tipos numéricos e coerção • 6. Operações matriciais • 7. Qualidade de arrays e valores ausentes • 8. Desempenho, memória e vetorização

Como estudar

Leia o princípio, execute o exemplo em um ambiente preparado, modifique entradas e resolva o desafio antes de avançar. Os exemplos são didáticos e não alteram os notebooks originais.

Relação com o acervo

Este guia é complementar à trilha de seis notebooks. A documentação oficial das bibliotecas é a fonte de referência para APIs e mudanças de versões.

01 Arrays, shape, dtype e eixos

ndarray armazena elementos com dtype comum, organizados por dimensões. shape descreve comprimentos dos eixos; ndim informa quantos eixos existem. Reduções com axis removem o eixo indicado, salvo keepdims=True.

Exemplo executável

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
assert a.shape == (2, 3)
assert a.mean(axis=0).tolist() == [2.5, 3.5, 4.5]
assert a.mean(axis=1).tolist() == [2., 5.]
```

Atenção técnica. axis=0 reduz linhas e retorna informação por coluna; não memorize apenas a frase, visualize o eixo removido.

Exercício de consolidação. Explique a dimensão de a.sum(axis=1, keepdims=True).

02 Views, copies e indexação

Slices simples frequentemente criam views que compartilham memória; indexação avançada/booleana geralmente produz cópias. Modificar views altera a matriz-base. Verifique arr.flags e np.shares_memory.

Exemplo executável

```
a = np.arange(6).reshape(2, 3)
v = a[:, :2]
assert np.shares_memory(a, v)
c = a[:, :2].copy()
assert not np.shares_memory(a, c)
```

Atenção técnica. Não confunda atribuição por máscara com o resultado de indexação avançada: as regras de cópia e atualização são diferentes.

Exercício de consolidação. Mostre como selecionar apenas valores maiores que 2.

03 Broadcasting e máscaras

No broadcasting, dimensões são alinhadas à direita; cada par deve coincidir ou conter 1. A expansão é lógica, evitando cópias integrais em muitos casos. Máscaras condicionais selecionam dados para análise.

Exemplo executável

```
x = np.arange(12).reshape(4, 3)
media = x.mean(axis=0)
centralizado = x - media
assert centralizado.shape == (4, 3)
assert x[x[:, 0] > 3].shape[1] == 3
```

Atenção técnica. Broadcasting não elimina o custo de um resultado intermediário; avalie memória em operações muito grandes.

Exercício de consolidação. Centralize as colunas e verifique se as médias ficam próximas de zero.

04 Aleatoriedade, precisão e qualidade

Use np.random.default_rng para simulações independentes do estado global. Registre sementes, versões e origem dos dados. Tipos inteiros podem sofrer overflow; funções agregadoras podem produzir NaN para valores ausentes.

Exemplo executável

```
rng = np.random.default_rng(42)
amostra = rng.normal(0, 1, size=(5, 3))
assert amostra.shape == (5, 3)
print(np.quantile(amostra, [0.25, 0.5, 0.75]))
```

Atenção técnica. Repetibilidade com uma semente não garante identidade de resultados entre versões e plataformas.

Exercício de consolidação. Compare média e mediana de amostras com valores extremos.

05 Tipos numéricos e coerção

ndarray é homogêneo, e operações entre dtypes podem promover o resultado. Inteiros com largura fixa podem sofrer overflow; floats possuem precisão limitada. O tipo final importa ao acumular grandes volumes ou converter dados tabulares.

Exemplo executável

```
import numpy as np
a = np.array([1,2,3], dtype=np.int64)
b = a.astype(np.float64)
assert b.dtype == np.float64
assert np.sum(a) == 6
```

Atenção técnica. Evite converter identificadores para float: valores inteiros grandes podem perder precisão.

Exercício de consolidação. Compare dtype e shape de arrays criados a partir de listas mistas.

06 Operações matriciais

O operador @ representa produto matricial; * é multiplicação elemento a elemento. Dimensões internas devem ser compatíveis no produto de matrizes. Transposição troca eixos, e np.linalg fornece ferramentas de álgebra linear.

Exemplo executável

```
a = np.array([[1.,2.],[3.,4.]])
v = np.array([1., 0.])
assert (a @ v).tolist() == [1., 3.]
assert (a * a).shape == (2,2)
```

Atenção técnica. Não use inversa explícita para resolver sistemas sem necessidade; prefira np.linalg.solve para Ax=b.

Exercício de consolidação. Verifique a diferença entre a*a e a@a.

07 Qualidade de arrays e valores ausentes

NaN indica valores não numéricos ou ausências em arrays de float. np.isnan e np.isfinite ajudam a filtrar; agregações especiais nanmean e nansum ignoram NaNs, porém não substituem avaliação dos mecanismos de ausência.

Exemplo executável

```
x = np.array([2., np.nan, 6.])
assert np.isnan(x).sum() == 1
assert np.nanmean(x) == 4.
assert np.isfinite(x).sum() == 2
```

Atenção técnica. Ignorar NaNs pode enviesar resultados; documente quantos valores foram excluídos e por quê.

Exercício de consolidação. Compare média comum e nanmean de um array com ausências.

08 Desempenho, memória e vetorização

Arrays NumPy evitam loops Python para operações numéricas homogêneas. Ganhos dependem de tamanho, tipo e memória. Broadcasting de grandes matrizes pode gerar temporários. Prefira medir com cenários representativos, sem extrapolar microbenchmarks.

Exemplo executável

```
x = np.arange(1000, dtype=float)
y = x**2 + 2*x
assert y.shape == x.shape
assert y[0] == 0
```

Atenção técnica. Vetorização não é sinônimo de baixo consumo de memória, especialmente em expressões encadeadas.

Exercício de consolidação. Meça tempo e memória de uma versão com laço e outra vetorizada.

Checklist de revisão

Verifique	Evidência
Conceito	Consegue explicar sem copiar o código?
Execução	O exemplo produz o resultado esperado?
Casos-limite	Entradas inválidas e extremas foram testadas?
Reprodutibilidade	O kernel e as dependências estão registrados?

Referências técnicas

Python: docs.python.org/3/ • NumPy: numpy.org/doc/stable/ • Matplotlib: matplotlib.org/stable/ • Pandas: pandas.pydata.org/docs/

Material didático autoral EP. A utilização de código e materiais de terceiros está sujeita às respectivas licenças.